

1. TEST

- 1. ¿Qué ocurre cuando hacemos `Cuenta c2 = c1;`?
 - **Respuesta correcta: c) Ambas referencias apuntan al mismo objeto.**
 - **Explicación:** En Java, las variables de objetos son *referencias* (punteros a la memoria). Al igualar una a la otra, no duplicas el objeto, solo creas un nuevo "mando a distancia" para controlar el mismo televisor. * 2. ¿Cuál es correcta respecto a `this`?
- 2. ¿Cuál es correcta respecto a `this`?
 - **Respuesta correcta: c) Hace referencia al objeto actual en ejecución.**
 - **Explicación:** Sirve para desambiguar cuando el parámetro de un método se llama igual que el atributo de la clase (ej. `this.saldo = saldo;`).
- 3. Si un atributo es `static`:
 - **Respuesta correcta: b) Es compartido por todos los objetos.**
 - **Explicación:** Pertenece a la clase, no a la instancia. Si un objeto modifica ese atributo estático, el cambio se refleja para todos los demás objetos de esa clase.
- 4. Si una excepción no es capturada:
 - **Respuesta correcta: c) Puede propagarse hasta main.**
 - **Explicación:** Si un método no captura la excepción con un `try-catch`, la "lanza" al método que lo llamó. Esto sube por la pila de llamadas hasta llegar al `main`. Si allí tampoco se captura, el programa "rompe" y termina.
- 5. ¿Qué ocurre al crear: `Cuenta[] cuentas = new Cuenta[3];`?
 - **Respuesta correcta: b) Se crean 3 referencias a null.**
 - **Explicación:** Típico fallo de examen. Al crear un array de objetos, solo creas las "cajas" para guardarlos, pero las cajas están vacías (`null`) hasta que hagas un `new Cuenta()`

2. EXPLICA CÓDIGO

Java

```
int[] a = {1, 2, 3}; // Se crea el array original en memoria. [cite: 135, 136]
int[] b = a;         // 'b' apunta a la MISMA memoria que 'a' (referencia). [cite: 137]
int[] c = a.clone(); // 'c' es un array NUEVO, pero con los mismos valores [1, 2, 3]. [cite: 138]
b[0] = 50;           // Como 'b' y 'a' son lo mismo, el array original pasa a ser [50, 2, 3]. [cite: 139]
c[1] = 100;           // 'c' es independiente, su array pasa a ser [1, 100, 3]. No afecta a 'a'. [cite: 140]
System.out.println(a[0] + " " + a[1]); // [cite: 141]
```

- **¿Qué se imprime por pantalla?** Se imprimirá: **50 2**
- **¿Y si se hace un print de b[3] qué ocurre?** Se lanzará una excepción **ArrayIndexOutOfBoundsException**. El array tiene tamaño 3 (índices 0, 1 y 2). El índice 3 está fuera de los límites.

3. ERRORES

El código propuesto tiene fallos graves de sintaxis y de buenas prácticas.

1. **Excepción no declarada:** El método retirar lanza un `new Exception()`. Al ser una excepción comprobada (checked), Java obliga a añadir `throws Exception` en la firma del método. Alternativamente, puedes lanzar una `IllegalArgumentException` (que es *unchecked* y no obliga a modificar la firma).
2. **Operación sin asignación:** La línea `saldo - cantidad;` realiza la resta, pero no guarda el resultado en ningún sitio. Debe ser `saldo = saldo - cantidad;` o `saldo -= cantidad;`.
3. **Lógica del constructor:** Si se pasa un saldo negativo al crear la cuenta, el `if (saldo >= 0)` se salta. Esto deja el saldo a 0.0 por defecto sin avisar al usuario. Lo correcto, como pide el enunciado, es lanzar una excepción ahí también.

Java

```
public class Cuenta {  
    private double saldo; // [cite: 147]  
  
    public Cuenta(double saldo) throws Exception { // Añadimos throws Exception [cite: 148, 159]  
        if (saldo < 0) {  
            throw new Exception("El saldo inicial no puede ser negativo"); // Controlamos error inicial [cite:  
149, 150, 159]  
        }  
        this.saldo = saldo;  
    }  
  
    public void retirar(double cantidad) throws Exception { // Añadimos throws Exception [cite: 152, 159]  
        if (cantidad > saldo) {  
            throw new Exception("Saldo insuficiente"); // [cite: 153, 154, 159]  
        }  
        saldo -= cantidad; // Corregimos la asignación matemática  
    }  
}
```

4. PREGUNTAS CORTAS

- **Explica por qué los atributos deben ser privados:** Para garantizar el principio de **encapsulación**. Al hacerlos privados, evitas que otras clases modifiquen los datos de forma incontrolada o asignen valores inválidos (ej. un saldo negativo o un DNI sin letra). Obligas a que cualquier modificación pase por métodos públicos (getters y setters), donde puedes programar validaciones.
- **Explica qué es una clase y qué es un objeto:** Una **clase** es el "molde" o la plantilla abstracta que define qué atributos (datos) y métodos (comportamientos) tendrán los elementos de ese tipo. Un **objeto** es una instancia concreta creada a partir de ese molde, con sus propios valores en memoria.

- Crea una clase sencilla y un objeto a partir de dicha clase:

Java

```
// CLASE (El molde)
public class Coche {
    private String marca;

    public Coche(String marca) {
        this.marca = marca;
    }
}

// OBJETO (La instancia en el main, por ejemplo)
Coche miVehiculo = new Coche("Toyota");
```